



工業4.0概論與 AIoT雲端物聯網

杜欣峯



工業 4.0 演進

回顧工業發展史，每一次的技術革命都深刻地重塑了生產模式與人類生活。從蒸汽機的發明到電力普及，再到電子資訊的興起，我們正處於一個前所未有的智慧製造轉型時代。圖 1-1 詳細展示了這一技術演進的里程碑。



圖 1-1 工業 4.0 的演進

資料參考：工業4.0的物聯網智慧工廠應用與實作/王進德

18世紀

19世紀

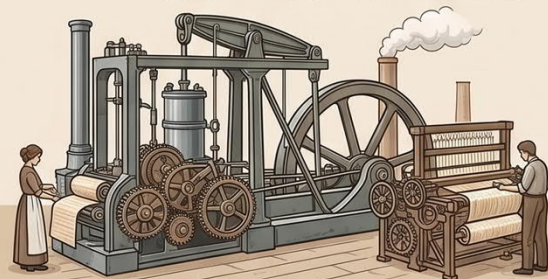
20世紀

20世紀初



工業 1.0 機械製造時代

時間段：18世紀60年代-19世紀中期

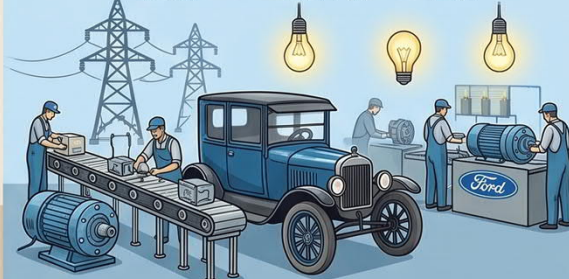


- ⚙️ 瓦特改良蒸汽機
- 🏭 機器力替代人力
- 👉 農業手工轉型到機械製造



工業 2.0 電氣化自動時代

時間段：19世紀後半期-20世紀初



- 💡 電力取代蒸汽
- 🚗 流水線生產
- 🏭 流水線生產
- 🏭 大規模生產能力



工業 3.0 電子資訊化時代

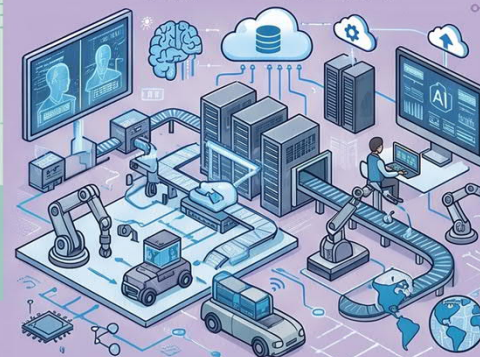
時間段：20世紀70年代-現在

PLC與電腦技術 製程自動化



工業 4.0 智慧聯網時代

時間段：2012年起（德國）

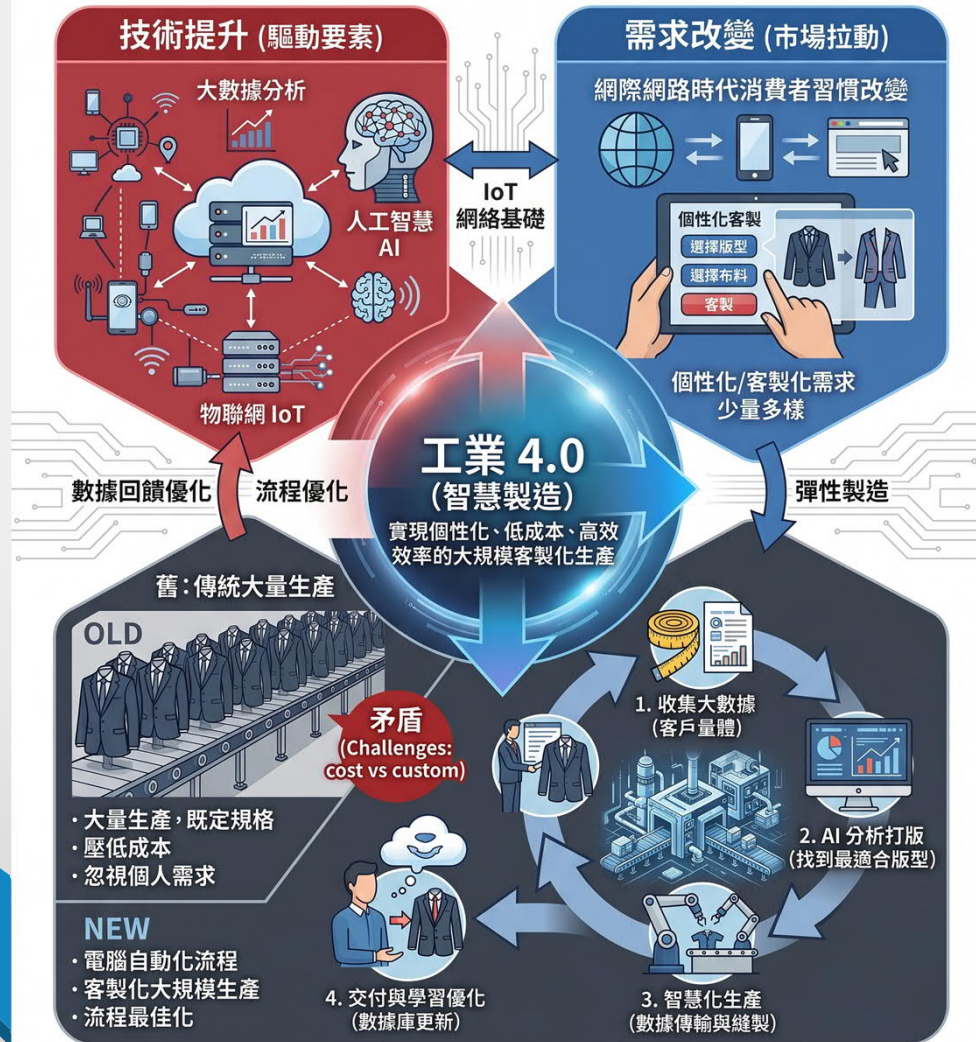


時間段：2012年起（德國）

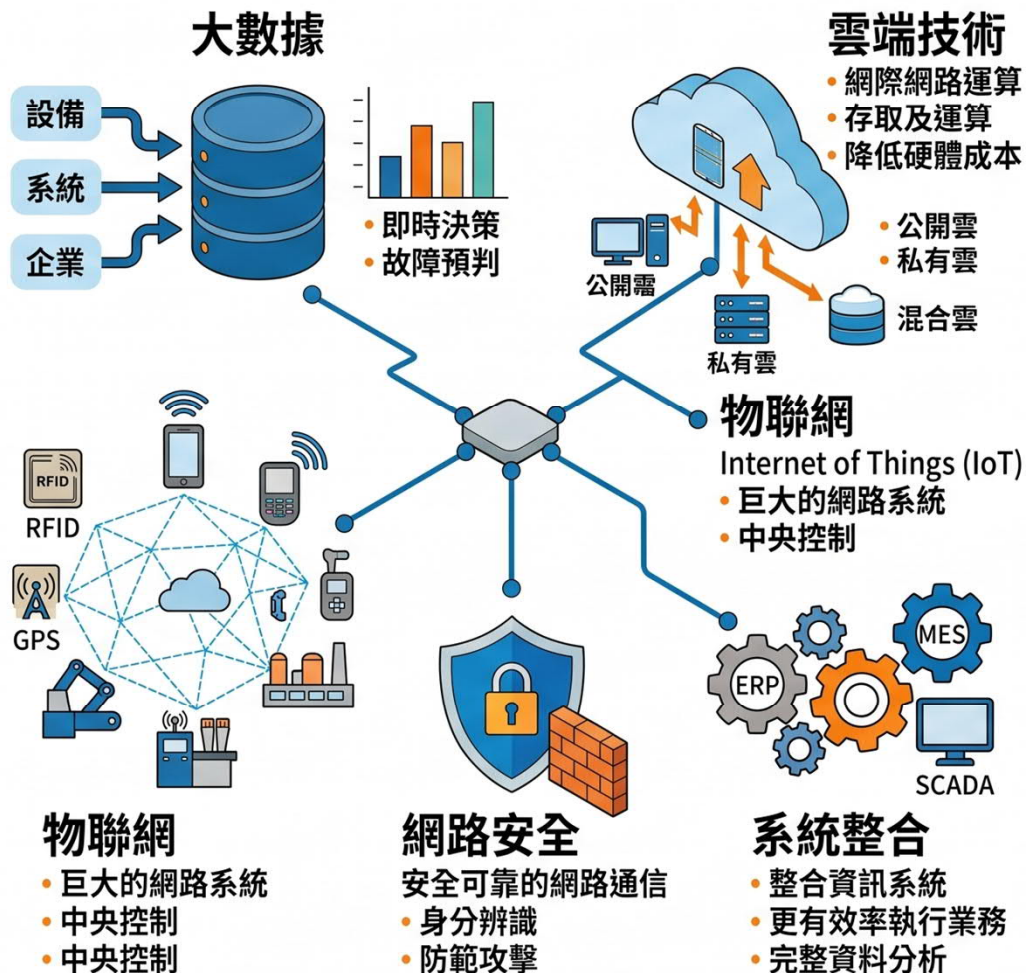
CPS（實體物理與虛擬網路融合系統） 自動化機器人 感測器物聯網（IoT） 供應鏈互聯網

**大數據分析 人機協作

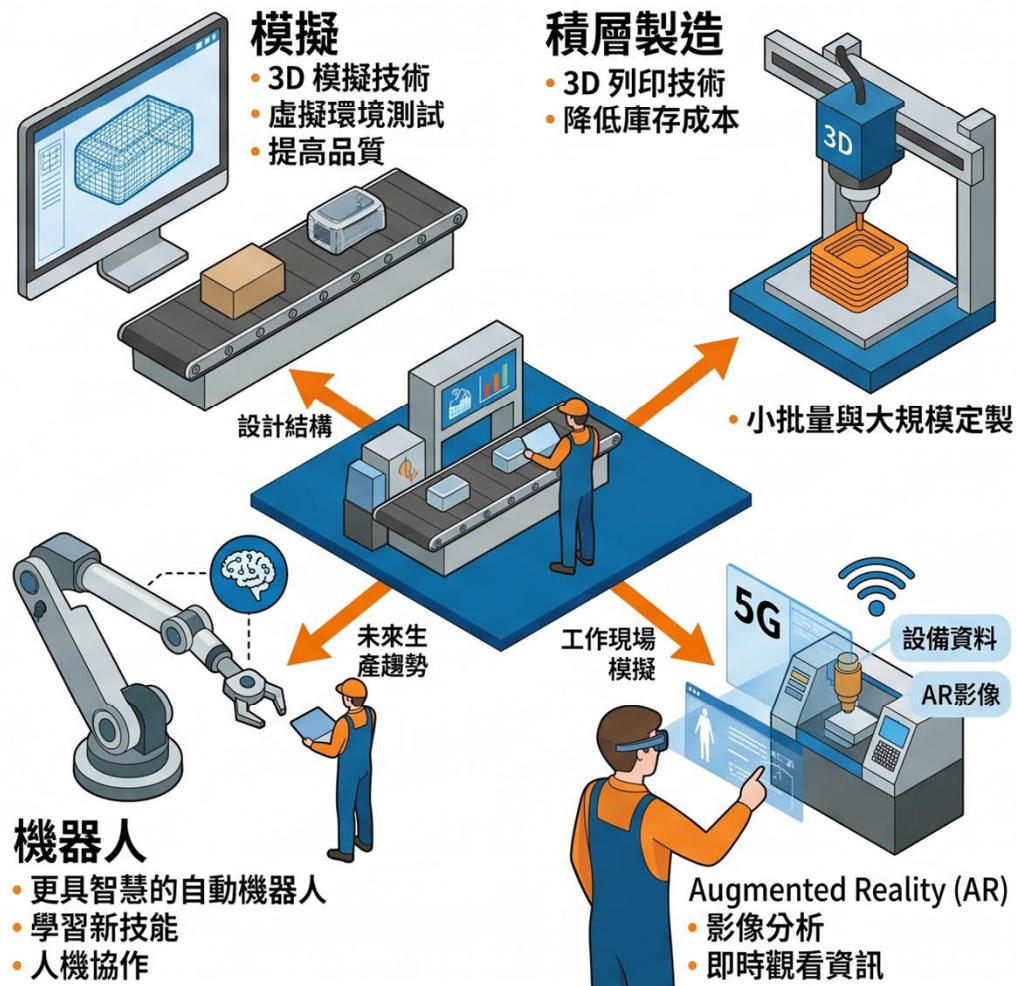
為何會有工業 4.0



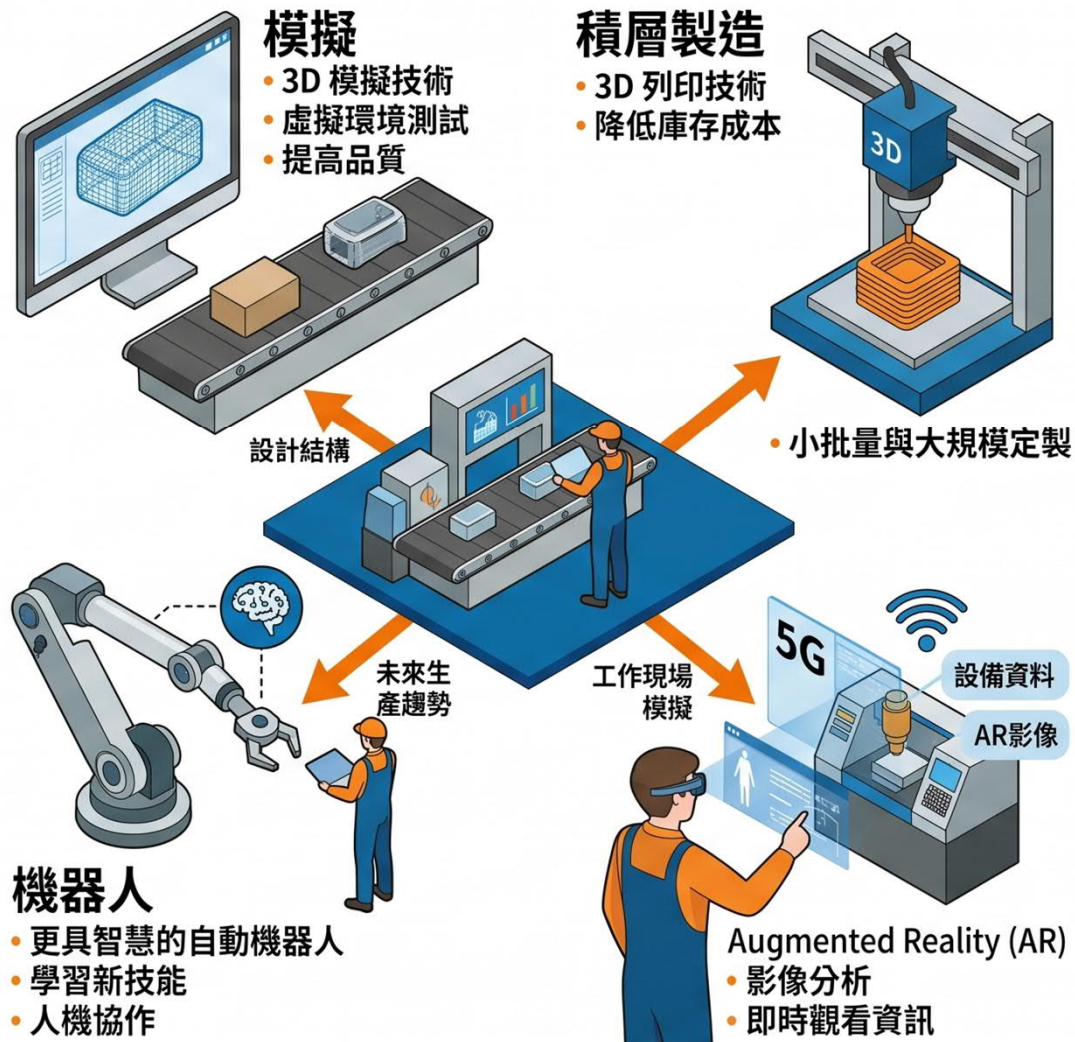
工業 4.0 九大科技 - 數據基礎與連接

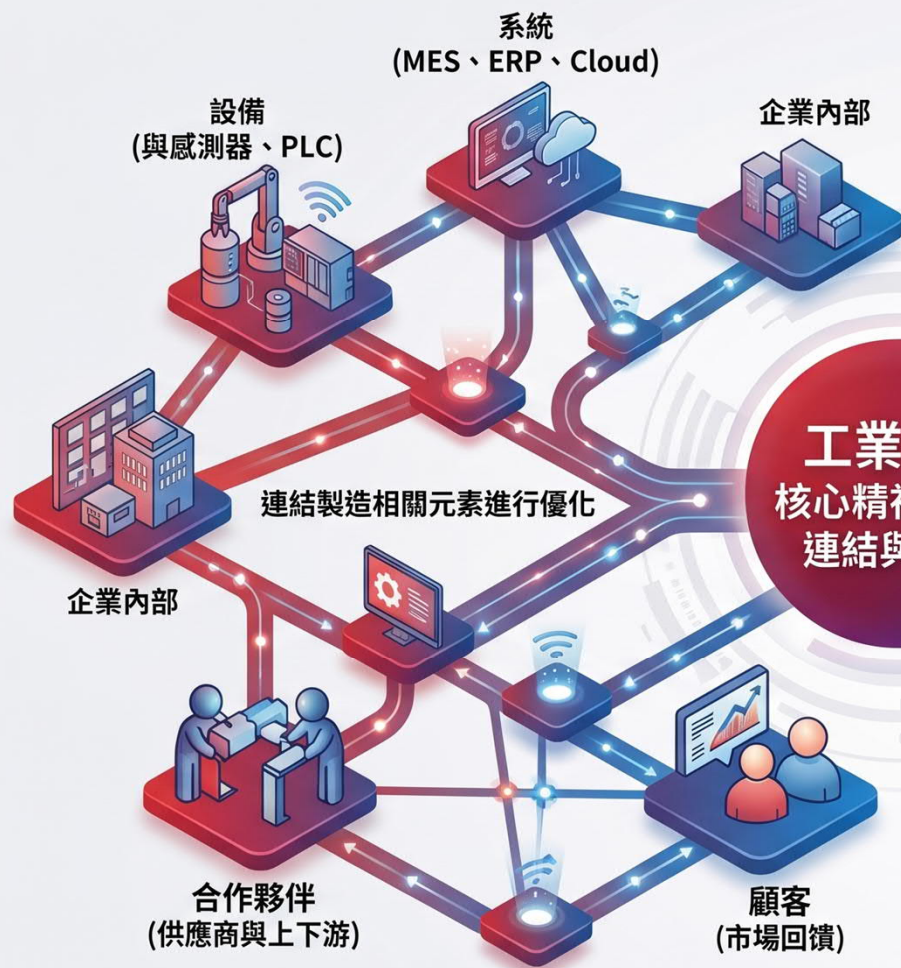


工業 4.0 九大科技 - 先進製造與優化

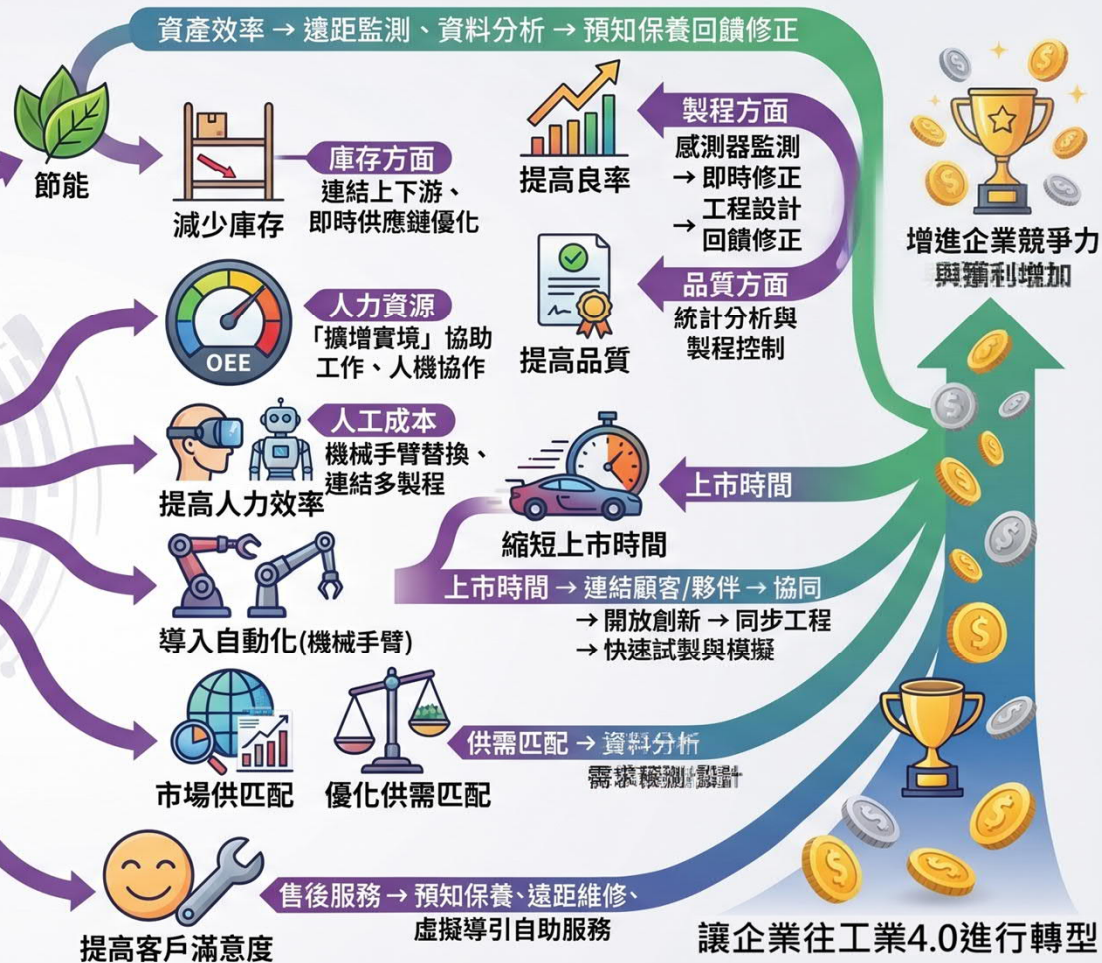


工業 4.0 九大科技 - 先進製造與優化



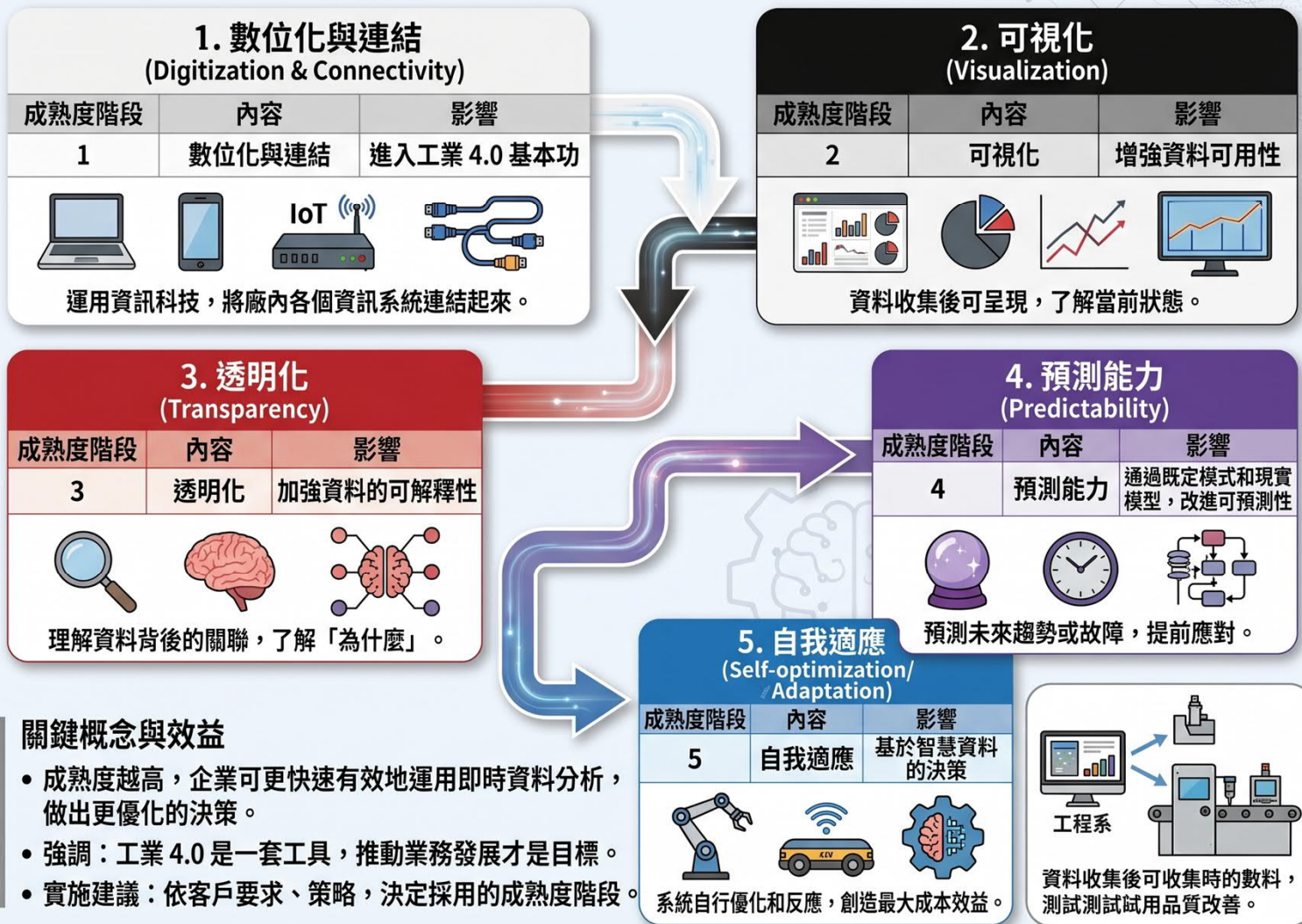


工業 4.0 核心精神圖解： 連結與優化

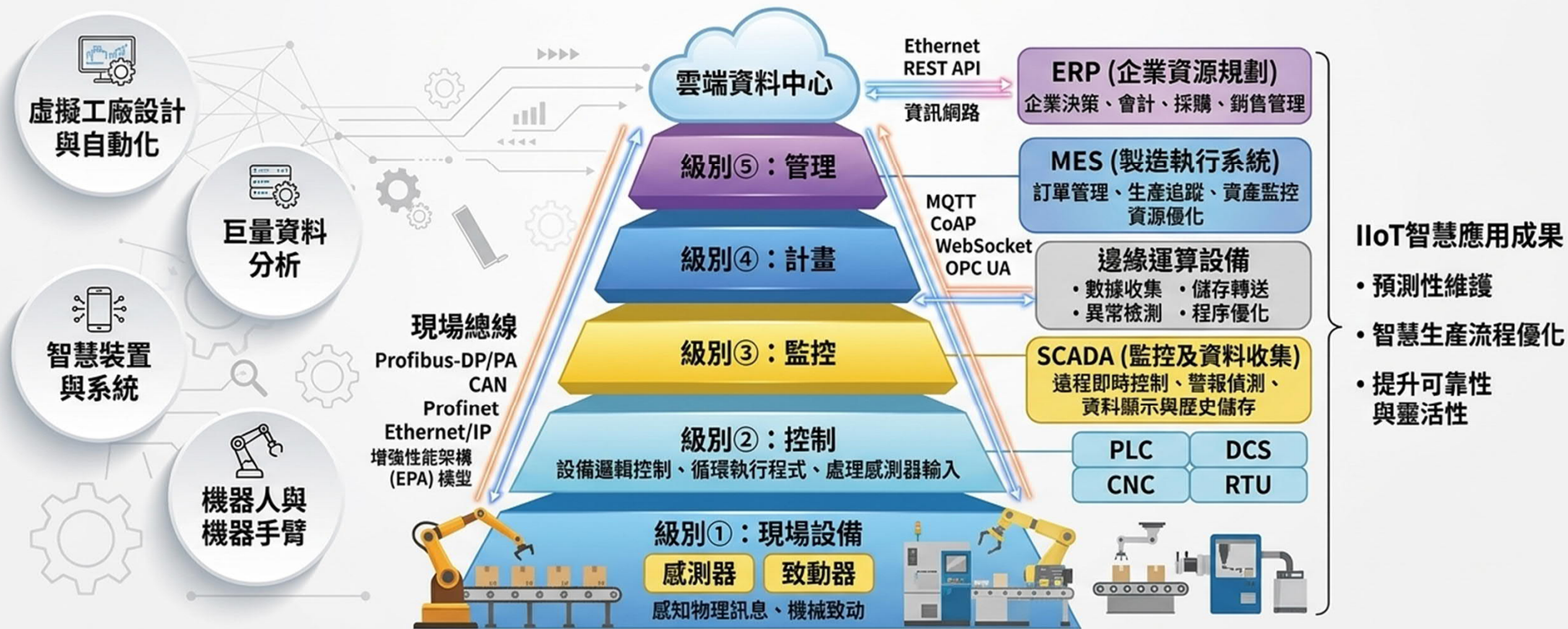


工業 4.0 成熟度發展路線圖 (由香港生產力促進局與德國 Fraunhofer IPT 合作訂定)

循序漸進優化，幫助企業依自身需求制定發展策略及目標。



工業物聯網 (IIoT) 架構與智慧應用圖解



AIoT (智慧物聯網) 架構與運作圖解

公理層 (ERP)

企業資源規劃 | 訂單管理、財務流程、採購、人員管理

計畫層 (MES)

製造執行系統 | 生產排程、資產監控、生產追蹤、品質管理

監控層 (SCADA)

監控與資料收集 | 遠端監控、人機介面 (HMI)、異常警報

控制層 (PLC)

控制與測量 | 可程式控制、RTU、CNC 控制器
<ETARL>
<GRUBB>

現場層 (裝置)

現場設備與感測器 | 製作、組裝、測試、測試、協調、致動



雲端與AI分析

時間序列
資料庫

分析模組

分析模組

資產註冊

大數據分析

IoT Hub

防火牆

<ISO/OSI-4>

AI (智慧)
分析大數據

IIoT 資料流

邊緣運算設備

- 數據收集
- 本地分析
- 異常檢測
- 邊緣開道

現場無線
資訊網路

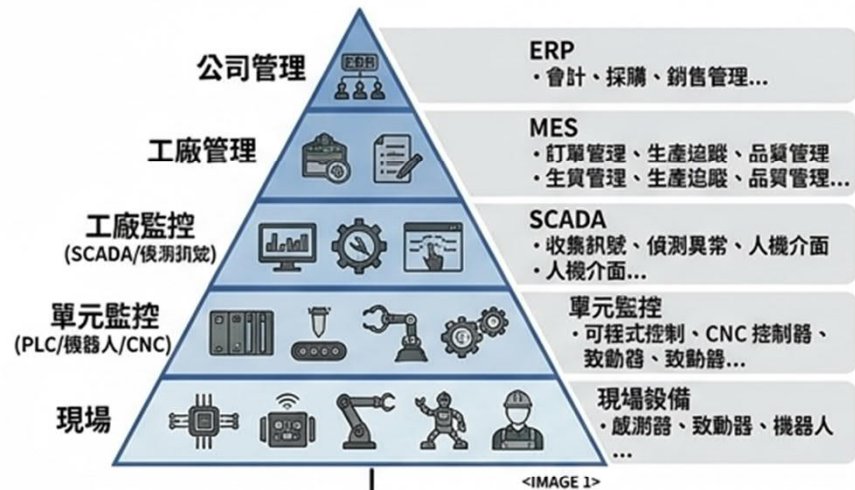


AI+IoT

AIoT (智慧物聯網)

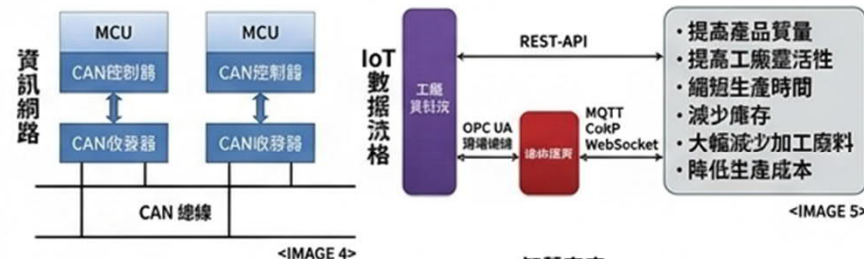
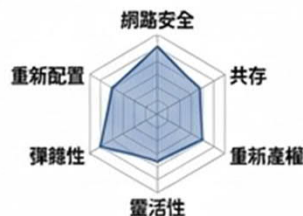
IoT數據上傳至雲端，
利用AI分析優化決策。

智慧工廠



智慧工廠特徵

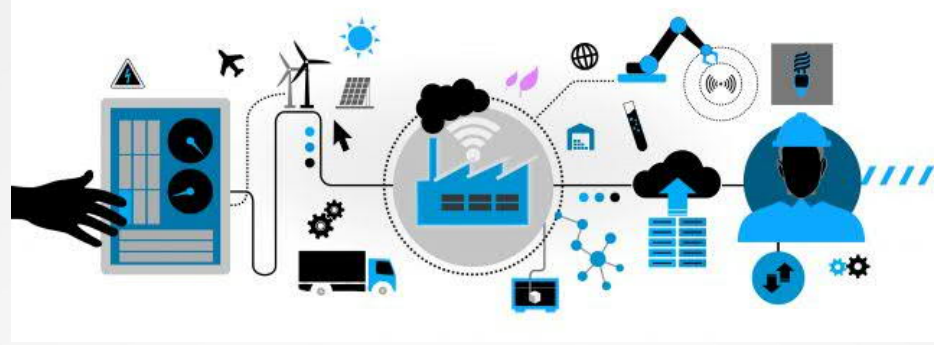
- 感測器網路
- 物聯網神經系統
- 善用機器人與機器手臂 (人機協作, 安全型)
- AIoT大腦 (IoT+AI=最佳化)



物聯網與智慧製造比較圖表

	 物聯網 (IoT)	 工業物聯網 (IIoT)	 人工智慧物聯網 (AIoT)	 智慧工廠
 核心焦點	  消費者使用、家屋、消費者	  製造效率、關關基礎講程	  智慧、數據分析析、主動行動	  全屬製造自動化、自產適應系統
 規模與網路	 小型規模、本地網路	 大規規模、千萬機备、企業	 綜合網路、智慧边缘設備	 完整植物系統、連接供給鏈
 技術組成	 基礎傳感器、連補連接續、雲	 工業協議、自動化性、分析化	 機機學習、边缘計算、電腦視覺	 大數據、Digital Twins, IIoT + AIoT
 主要應用	 远程控制、數據收取及續集	 流程優化、預測維護	 自治學習、優化、智能安全	 高效率工廠、全參生產協調





Node-RED 程式設計

Node-RED 介紹

作為物聯網 (IoT)
的一部分



基於流程的可視
視化開發工具

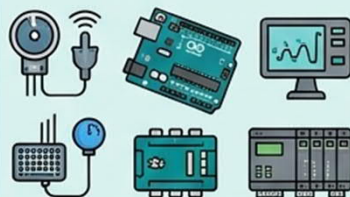


起源



最初由 IBM 開發

連接硬體設備



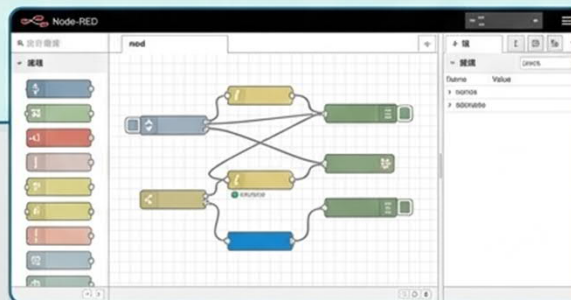
連接在線服務



連接 API
(應用程式介面)



基於 Web 瀏覽器的流程編輯器



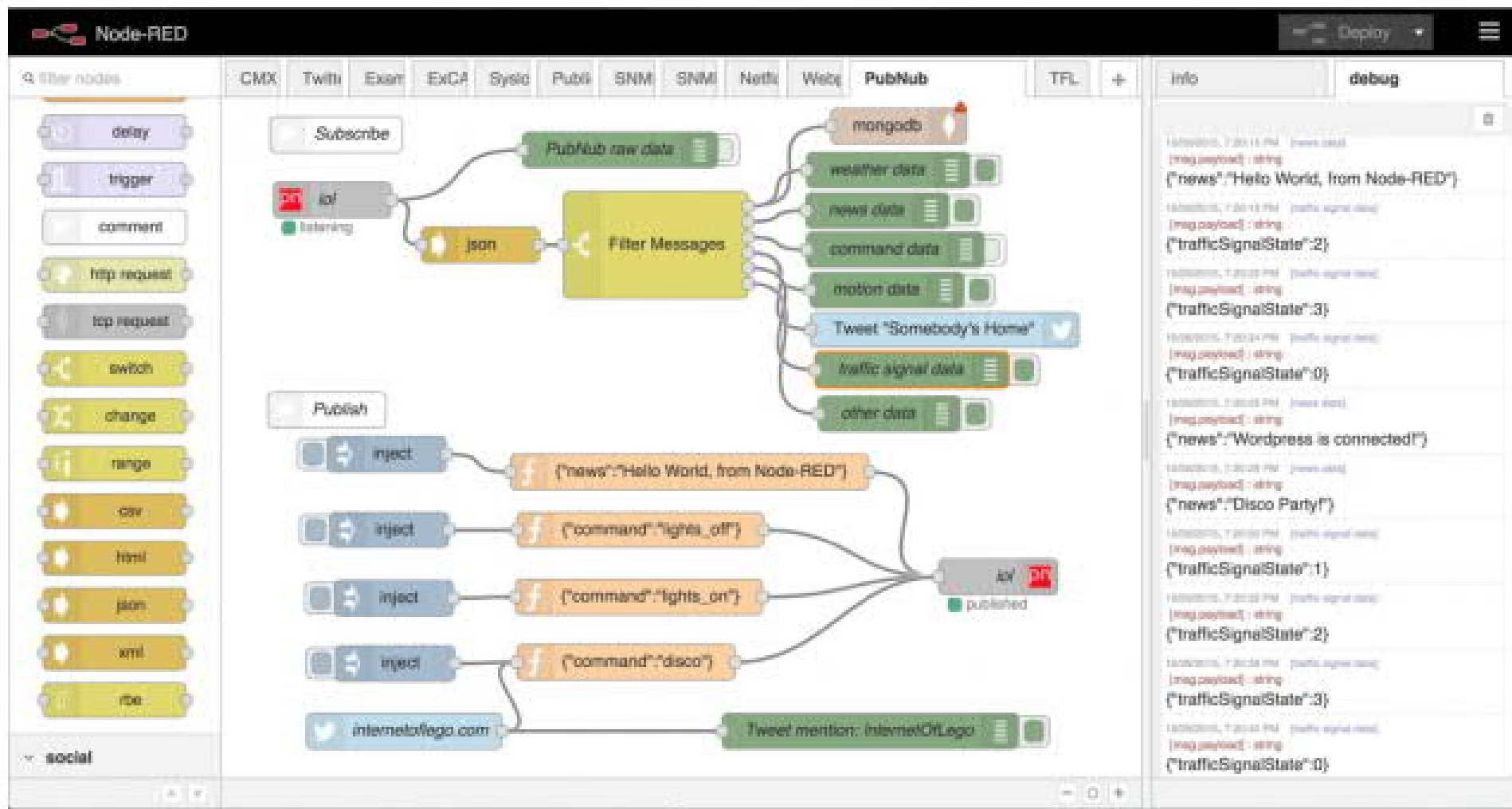
運行時技術
基於 Node.js 構建



流程儲存
使用 JSON 檔案儲存流程

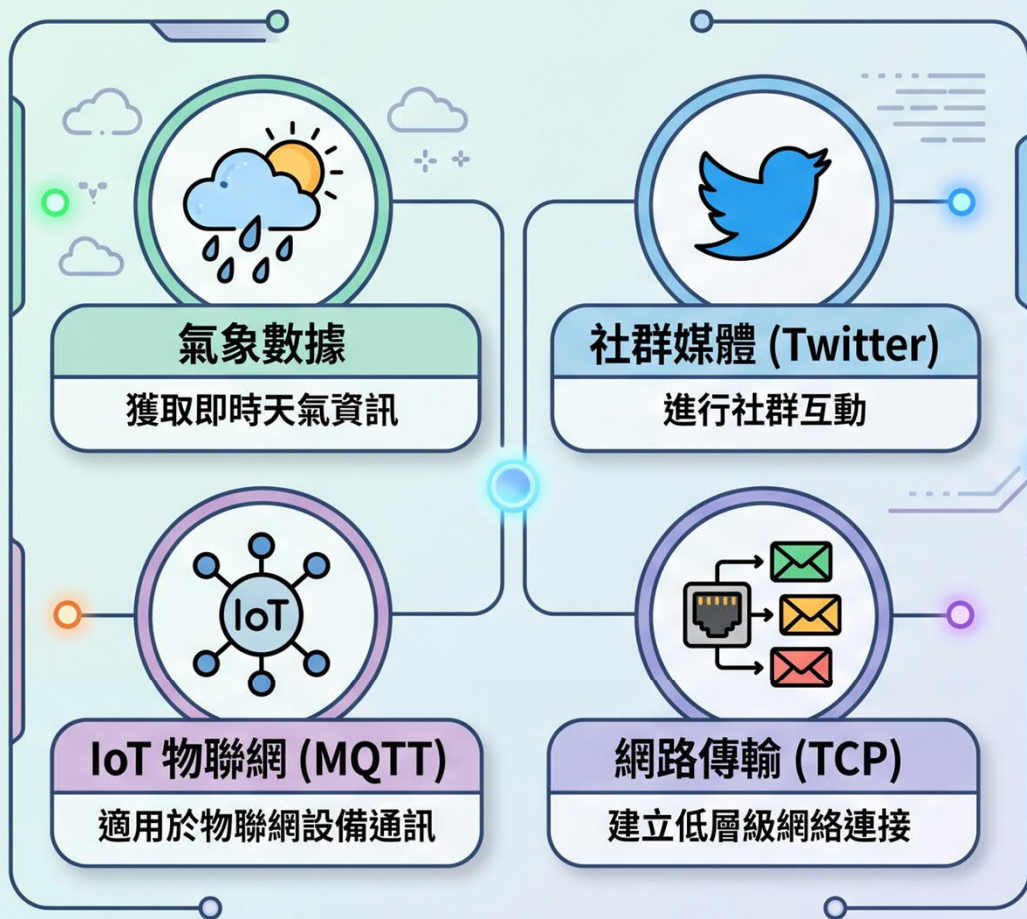


Node-RED 程式畫面



適合IoT應用的視覺化開發工具Node-RED

Node-RED 特色



1



圖形化介面快速開發
拖拽節點，快速組合流程，
節省開發時間。
快速原型製作。

2



適合概念驗證 PoC
迅速建立並測試構思，
降低風險與成本。
驗證核心功能。

3



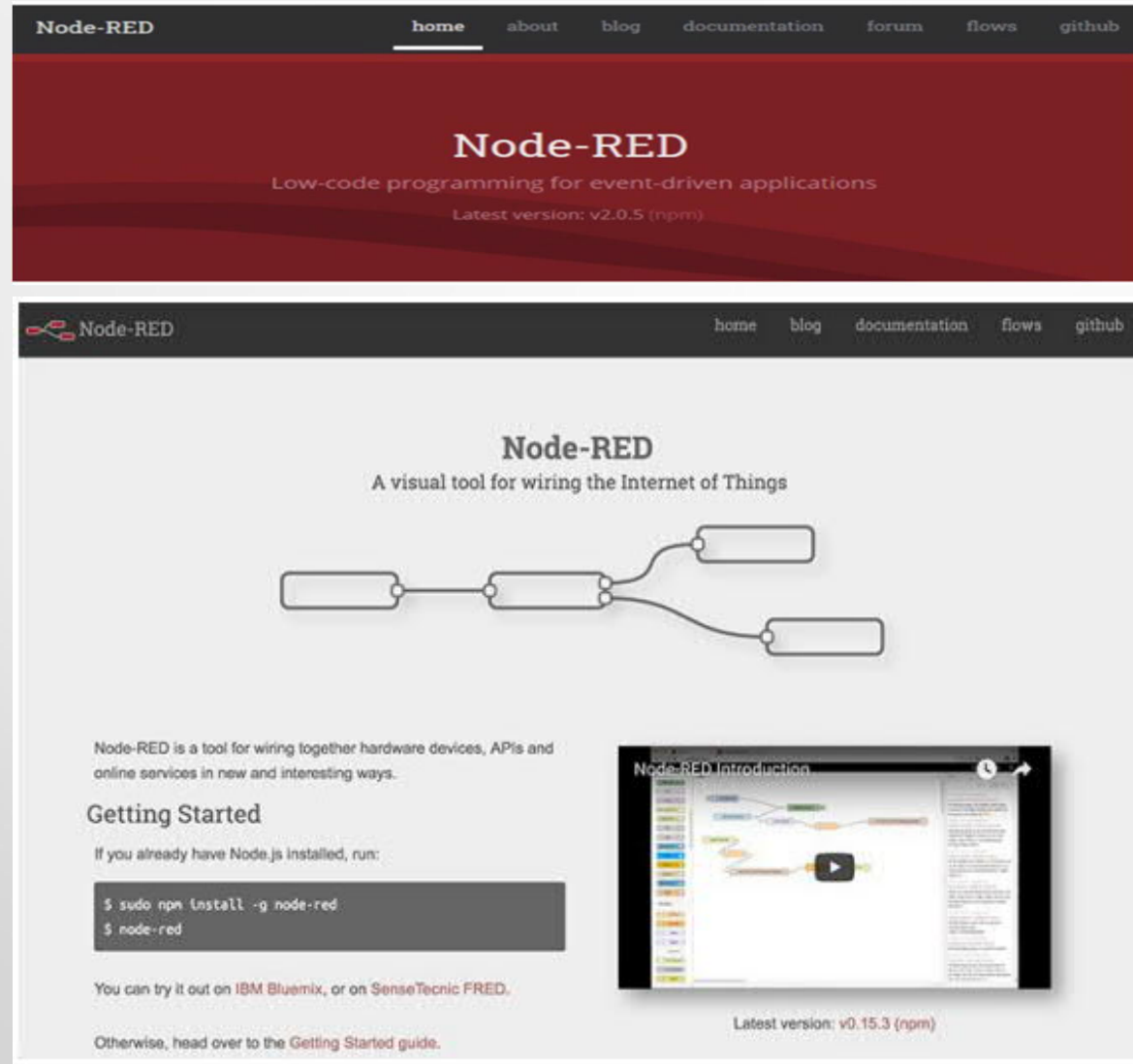
淺顯易懂附帶說明
完整節點說明與教學資源，
易於學習與上手。
豐富範例。

4



高度擴充性豐富資源
擁有龐大的函式庫與節點
社群，輕鬆整合各式服務。
高度自定義。

Node-RED 官方網站：<https://nodered.org/>



The image displays two screenshots of the Node-RED website. The top screenshot shows the main homepage with a dark red header and a large red banner. The bottom screenshot shows a detailed view of the Node-RED interface, including a visual flow diagram, a 'Getting Started' section with installation instructions, and a video player.

Node-RED
Low-code programming for event-driven applications
Latest version: v2.0.5 (npm)

Node-RED
A visual tool for wiring the Internet of Things

Node-RED is a tool for wiring together hardware devices, APIs and online services in new and interesting ways.

Getting Started

If you already have Node.js installed, run:

```
$ sudo npm install -g node-red  
$ node-red
```

You can try it out on IBM Bluemix, or on SenseTechnic FRED.

Otherwise, head over to the [Getting Started guide](#).

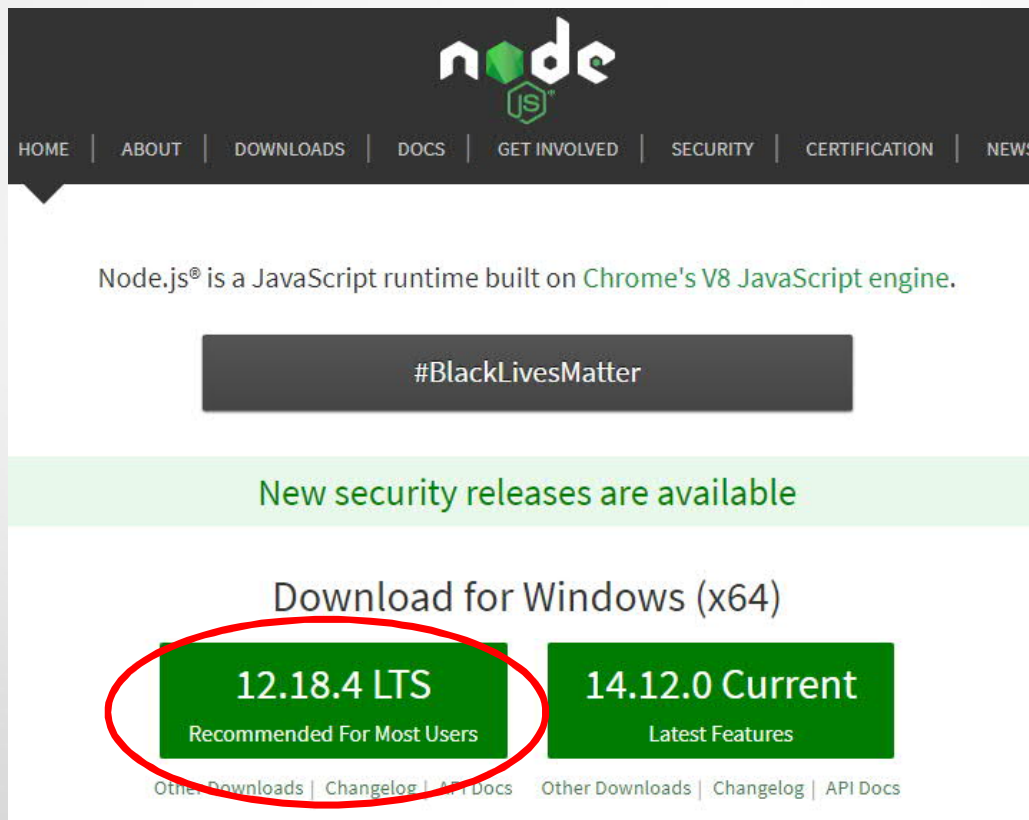
Node-RED Introduction

Latest version: v0.15.3 (npm)

Node-RED 程式安裝

<https://nodejs.org/en/>

Node-Red以 Node.js 為基礎，所以要使用前必須要安裝 Node.js，直接前往 Node.js 官方網站下載安裝。



Node-RED 程式安裝

於命令提示字元(CMD)下輸入：
npm install -g node-red

npm

```
l>npm install -g node-red
```

```
.....] \ rollbackfailedoptional: verb npm-session 57d08544d9ee6ad0
```

Node-RED 程式啟動 (需常駐)

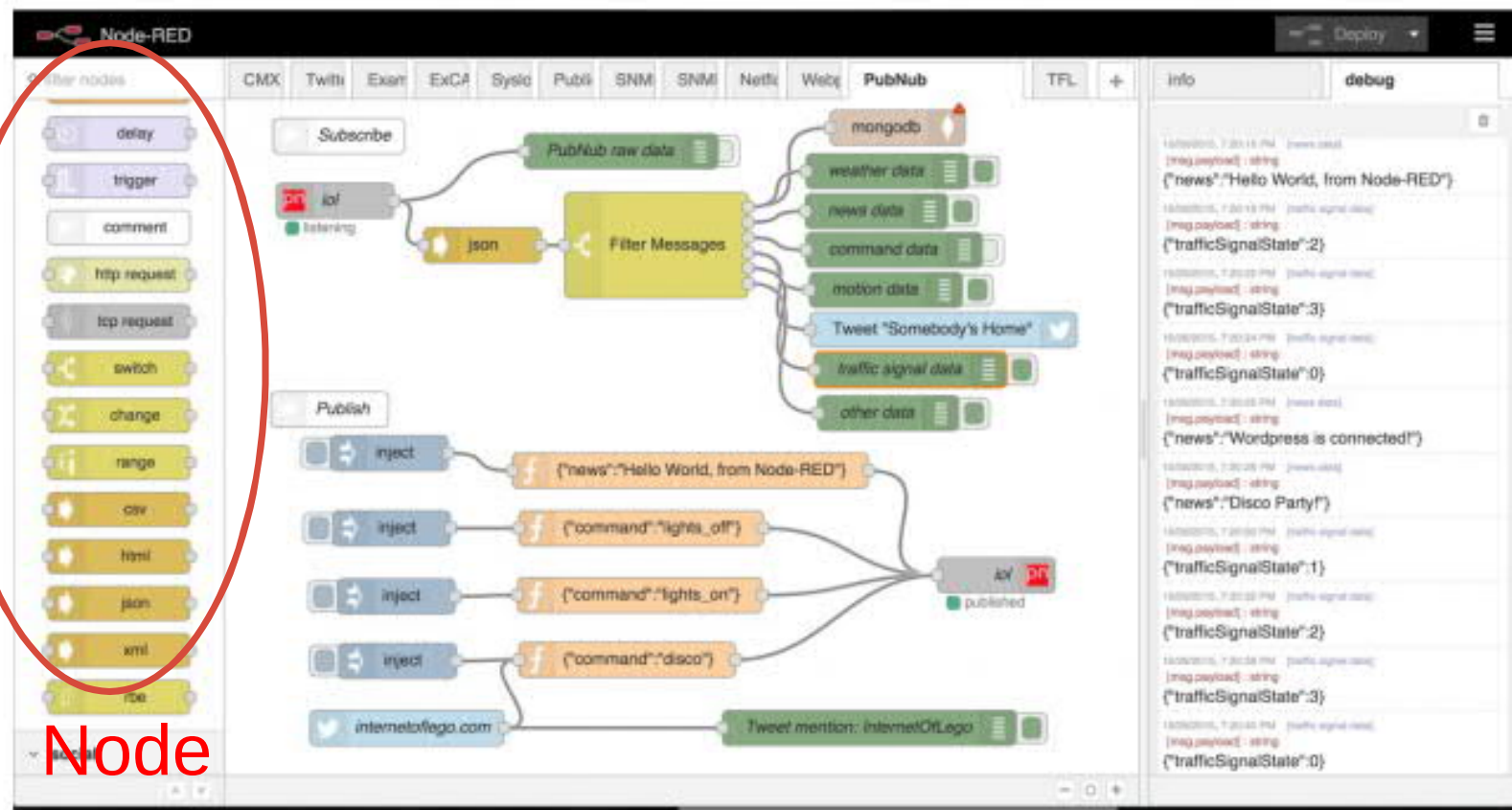
於命令提示字元下輸入：

node-red

```
Welcome to Node-RED
-----
29 Sep 00:27:44 - [info] Node-RED version: v1.1.3
29 Sep 00:27:44 - [info] Node.js version: v12.18.3
29 Sep 00:27:44 - [info] Windows NT 10.0.18362 x64 LE
29 Sep 00:27:55 - [warn] -----
29 Sep 00:27:55 - [info] Settings file : C:\Users\Noah-ideapad\.node-red\settings.js
29 Sep 00:27:55 - [info] Context store : 'default' [module=memory]
29 Sep 00:27:55 - [info] User directory : C:\Users\Noah-ideapad\.node-red
29 Sep 00:27:55 - [warn] Projects disabled : editorTheme.projects.enabled=false
29 Sep 00:27:55 - [info] Flows file : C:\Users\Noah-ideapad\.node-red\flows_DESKTOP-00T3VIC.json
29 Sep 00:27:55 - [info] Server now running at http://127.0.0.1:1880/
29 Sep 00:27:55 - [warn]
```

打開瀏覽器，網址列輸入 <http://127.0.0.1:1880/>
，就可以打開 Node-RED ，

INPUT+FUNCTION+OUTPUT



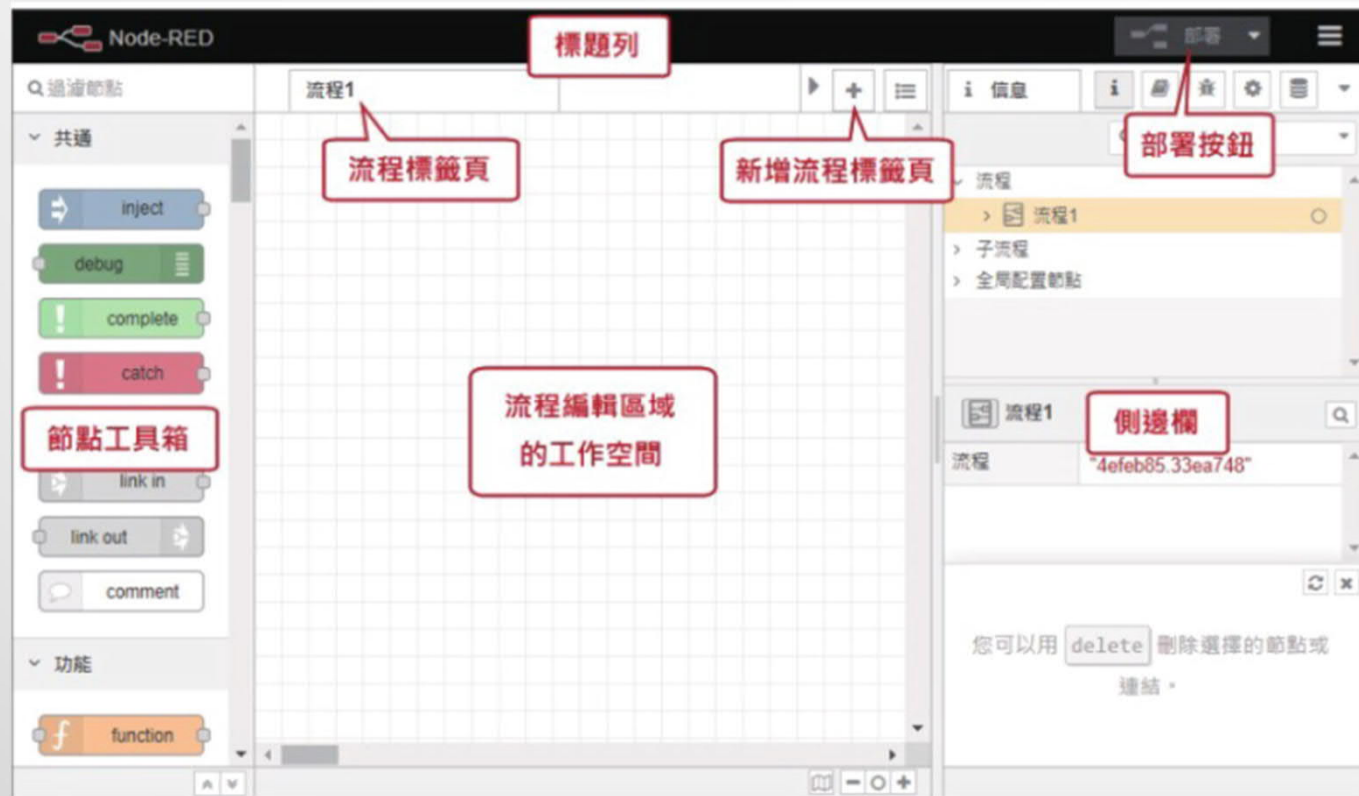
Node

適合IOT應用的視覺化開發工具Node-RED

flow= node + connector

<https://nodered.org/>

Node-RED 編輯器的使用介面說明



程式匯出

匯出節點至剪貼簿

匯出

已選擇的節點

現在的節點

所有流程

剪貼簿

庫

```
[{"id":"cc9b6663.131358","type":"tab","label":"例題8-1_溫度亂數","disabled":false,"info":""}, {"id":"ce0ddc14.d2502","type":"inject","z":"cc9b6663.131358","name":"","props":[{"p":"payload"}, {"p":"topic","vt":"str"}],"repeat":"","crontab":"","once":false,"on ceDelay":0.1,"topic":"","payload":"","payloadType":"date","x":120,"y":160,"wires":[["a2bcbb40.b143d8"]]}, {"id":"a2bcbb40.b143d8","type":"random","z":"cc9b6663.131358","name ":"亂數溫 度","low":"15","high":"35","inte":"false","property":"payload","x": 280,"y":220,"wires":[["720a603e.0457d","d955a5.d9967a58"]]}, {"id":"720a603e.0457d","type":"switch","z":"cc9b6663.131358","name ":"","property":"payload","propertyType":"msg","rules": [{"t":"gte","v":"28","vt":"str"}, {"t":"lte","v":"20","vt":"str"}, {"t":"else"}],"checkall":"true","repair":false,"outputs":3,"x":280,"y":320,"wires":[["ba89c0d4.c45e1"],["34779ffa.b43f5"], ["e6c394d0.945578"]]}, {"id":"ba89c0d4.c45e1","type":"function","z":"cc9b6663.131358","nam e":"Hot","func":"msg.payload = \"It's hot. Turn on AC\\\"\\nreturn msg;","outputs":1,"noerr":0,"initialize":"","finalize":"","x":450,"
```

緊湊

已格式化

取消

下載

匯出到剪貼簿

部署

三

顯示

匯入

匯出

搜尋流程

修改節點配置

流程

子流程

組

節點管理

設置

鍵盤快速鍵

Node-RED website

v1.1.2

程式匯入

匯入節點

剪貼簿

庫

例子

在這裡粘貼節點

匯入所選檔案

匯入到

現在的節點

新流程

取消

匯入



基本設定

使用者設定

關閉

顯示

鍵盤

Palette

語言

瀏覽器默認

格線

☒ 顯示格線

☒ 對齊格線

格線尺寸

20

節點

☒ 顯示節點狀態

☒ 顯示新添加節點的標籤

其他

☐ 顯示小提示



節點資料庫管理

使用者設置

顯示

鍵盤

Palette

節點

過濾節點

node-red

1.1.2

> 46 個節點

使用中

node-red-node-mysql

0.1.0

> 2 個節點

移除

全部禁用

node-red-node-random

0.2.0

> 1 個節點

使用中

node-red-node-rbe

0.2.9

> 1 個節點

全部禁用

node-red-node-tail

0.1.1

> 1 個節點

全部禁用

關閉

部署

顯示

匯入

匯出

搜尋流程

修改節點配置

流程

子流程

組

節點管理

設置

鍵盤快速鍵

Node-RED website

v1.1.2

節點擴充資料庫

使用者設置

顯示 節點 安裝

鍵盤 排序: a-z順序 日期順序

Palette

Q ard

116 / 2780

- @arduino/node-red-contrib-arduino-iot-cloud
Node-RED nodes to talk to Arduino IoT Cloud
1.0.6 2月前 安裝
- @hardwario/node-red-contrib-hardwario
Set of helpful examples to get you started with the multifunctional industrial HARDWARIO IoT Kits.
1.0.1 7月前 安裝
- @hardwario/node-red-contrib-hardwario-voice
This node enables you to better control the multifunctional industrial HARDWARIO IoT Kits with support for Google Assistant
1.1.4 6月前 安裝
- cn-dashboard-nodes
Install
0.0.2 2年, 1月前 安裝

部署

顯示

匯入
匯出

搜尋流程

修改節點配置

流程
子流程
組

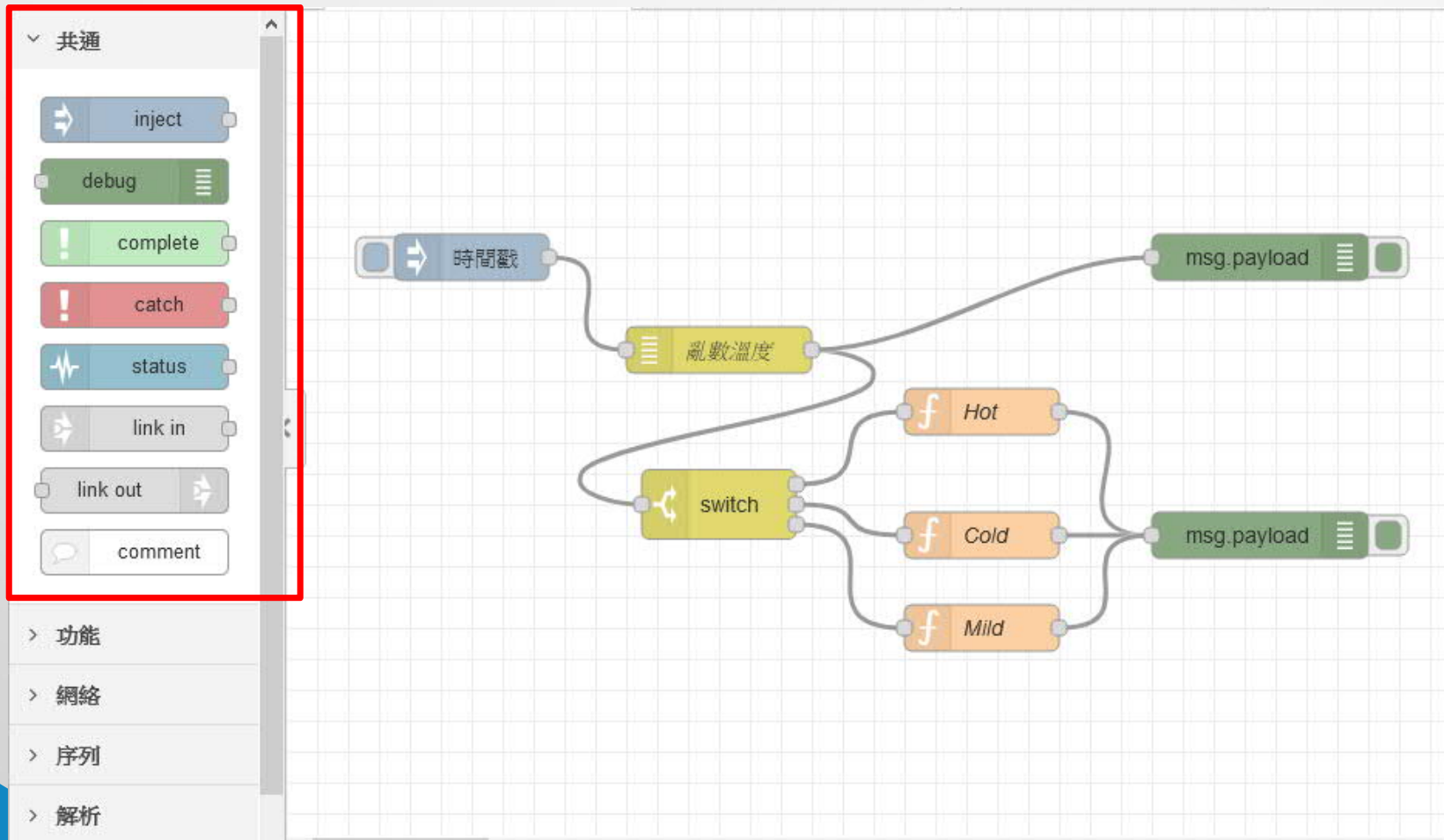
節點管理

設置

鍵盤快速鍵
Node-RED website
v1.1.2

節點基本功能介紹

共通(Common):基本流程的起始、狀態和除錯的相關節點。



共通-Inject

▼ 共通

- inject
- debug
- complete
- catch
- status
- link in
- link out

時間戳

inject

手動或定期得將消息注入流程中。消息的有效荷載可以為多種類型，包括字符串，JavaScript對象或當前時間。

▼ 輸出

payload various

指定的消息的有效荷載。

topic 字符串

可以在節點中配置的可選屬性。

編輯 inject 節點

刪除

取消

完成

屬性

名稱

名稱

≡

msg. payload

=

時間戳記

×

≡

msg. topic

=

msg.

×

flow.

global.

文字列

數字

布林

{ } JSON

二進位流

時間戳記

expression

\$ env variable

+ 添加

☐ 立刻執行

出後

重複

無

☐ 有效

共通-debug

debug

在“調試”側邊欄選項卡和運行時日志中顯示選定的消息屬性。默認情況下，它會顯示 **msg.payload** 的值，但您也可以將其設置成顯示任意屬性，完整消息或JSONata表達式的結果。

The screenshot shows the '編輯 debug 節點' (Edit Debug Node) dialog box in a Node-RED environment. The background shows a canvas with a node labeled 'msg.payload' and a sidebar with '亂數' and '例題8-1_溫度亂數'.

編輯 debug 節點

刪除 取消 完成

屬性

輸出

目標 ☒ 除錯窗口 ☐ Console ☐ 節點狀態 (32位元字元)

名稱



inject 節點送出的 Unix 時間戳記，表示
現在時間與1970年1月1日00:00:00的時間差，
計算單位為毫秒 (ms)

線上時間戳記轉換網址：
<https://www.epochconverter.com/>



```
{
  "_msgid": "1b7d51eaf89c9b3b",
  "payload": 1711096460475,
  "topic": ""
}
```

msg 物件是擁有3 個屬性的 JavaScript 物件,其說明如下:

_msgid: 訊息識別的唯一名稱。

payload: 在節點之間的資料傳遞是使用 payload 屬性,這也是一定會被下一個節點讀取的屬性,傳遞資料可以是物件、日期、布林、字串和數值等。

topic: 文字內容的訊息主題。

msg 物件是預設內容,擁有3個屬性,部分 Node-RED 節點送出的訊息會自行新增專屬屬性,例如:mqtt in 節點會新增 qos 和 retain 屬性

建立 msg 物件內容

msg. payload	=	▼ % 100	✕
msg. topic	=	▼ % score	✕
msg. age	=	▼ % 20	✕

score : msg : Object

```
▶ { _msgid: "8ecc7bea.304938", payload: 100, topic: "score", age: 20 }
```

延時觸發事件

☒ 立刻執行於 秒後, 此後

🔄 重複

周期定時觸發事件

☐ 立刻執行於 秒後, 此後

🔄 重複

每隔

➡

2021/6/12 下午2:38:47 node: 6866ece6.3b1964
msg.payload : number
1623479927708

2021/6/12 下午2:38:49 node: 6866ece6.3b1964
msg.payload : number
1623479929716

2021/6/12 下午2:38:51 node: 6866ece6.3b1964
msg.payload : number
1623479931725

固定時間的周期觸發事件

☐ 立刻執行於 秒後, 此後

☒ 重複 指定時間段並週期性執行

每隔 分鐘

介於 至

在 ☒ 星期一 ☐ 星期二 ☒ 星期三
☐ 星期四 ☒ 星期五 ☐ 星期六
☐ 星期天

固定時間觸發事件

☒ 重複 指定時間

在

在 ☒ 星期一 ☐ 星期二 ☐ 星期三
☐ 星期四 ☐ 星期五 ☐ 星期六
☐ 星期天

練習：

編輯 inject 節點

刪除 取消 完成

屬性

名稱 名稱

msg.payload = a_z Hello World|

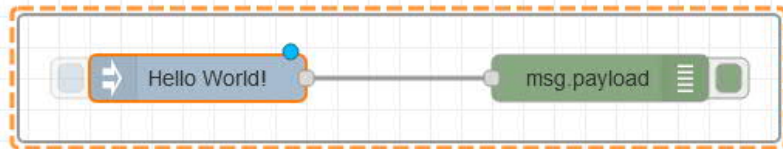
msg.topic = a_z

+ 添加 inject now

☐ 立刻執行於 0.1 秒後, 此後

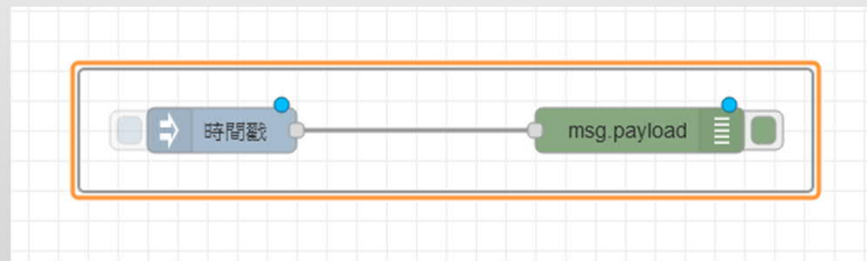
重複 無

☐ 有效



```
2020/8/7 上午8:55:08 node: e4dcfd9e.2b5428
msg.payload : string[12]
"Hello World!"
```

功能介紹-群組



共通-complete

complete

當另一個節點完成對消息的處理時觸發流程。

亂數

例題8-1_溫度亂數

完成: 1 個節點

編輯 complete 節點

刪除

取消

完成

屬性

選擇節點...

Q

☐ msg.payload

debug

☐ 123:Hello!

inject

☒ Print

debug

☐ msg.OK

debug

功能-function



可使用JavaScript
程式語言編寫函數

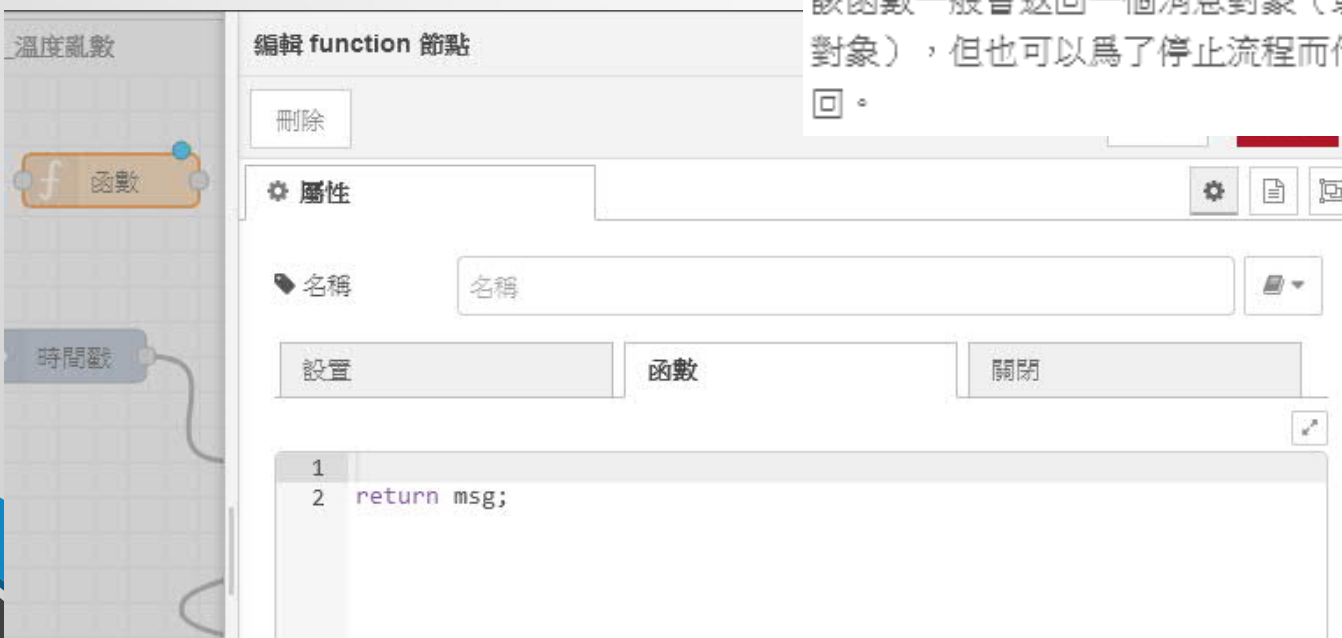
function

定義對接收到的消息進行處理的JavaScript代碼（函數的主體）。

輸入消息在名為`msg`的JavaScript對象中傳遞。

通常，`msg`對象將消息正文保留在`msg.payload`屬性中。

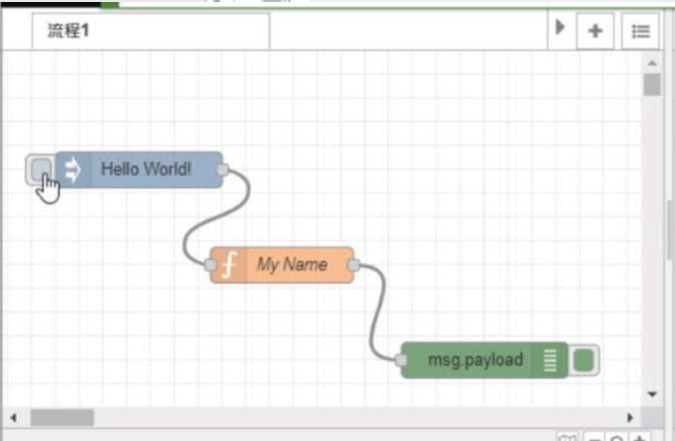
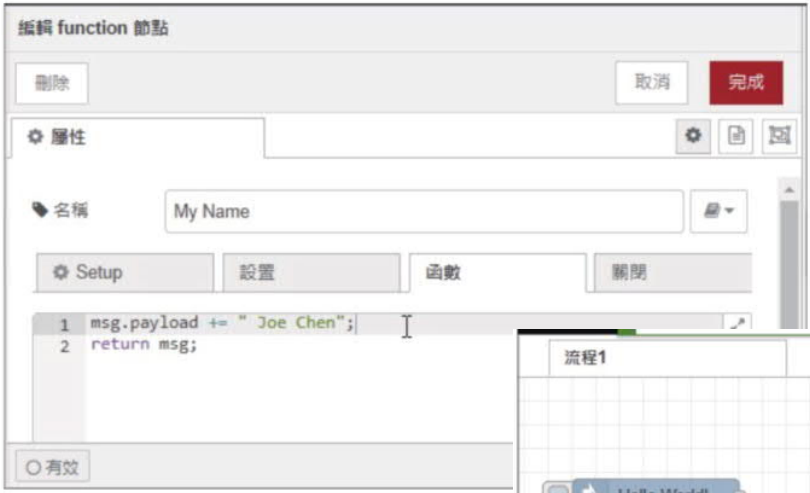
該函數一般會返回一個消息對象（或多個消息對象），但也可以為了停止流程而什麼都不返回。



練習：

在「編輯 function 節點」對話方塊的【名稱】欄輸入節點名稱【My Name】後,在下方輸入JavaScript 程式碼將訊息 msg.payload 使用「+=」運算子加上姓名字串,然後按【完成】鈕,如下所示:

```
msg.payload += " Joe Chen";  
return msg;
```

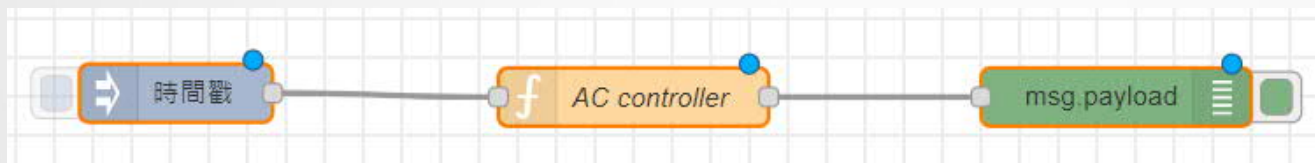


Time	Node	Payload
2021/6/11 下午9:05:19	node: 96bc7b85-b082d8	msg.payload : string[21] "Hello World! Joe Chen"
2021/6/11 下午9:05:25	node: 96bc7b85-b082d8	msg.payload : string[21] "Hello World! Joe Chen"
2021/6/11 下午9:05:44	node: 96bc7b85-b082d8	msg.payload : string[21] "Hello World! Joe Chen"

練習：

使用Function 節點做以下工作

1. 數字 <20 ，顯示溫度很冷，關掉風扇與冷氣
2. $28 >$ 數字 >20 ，顯示溫度剛好，開風扇、關掉冷氣
3. 數字 >28 ，顯示溫度很熱，開風扇與冷氣



屬性

名稱 AC controller

Setup 設置 函數 關閉

```
1 var temp = Number (msg.payload)
2 if (temp > 28) {
3   msg.payload = "It's hot. Turn on AC";
4 }
5 else if (temp < 20) {
6   msg.payload = "It's cold. Turn off AC and fan.";
7 }
8 else {
9   msg.payload = "It's nice. Turn off AC, but keep fan running";
10 }
11 return msg;
```

功能-random (擴充資料庫)

The screenshot displays the Node-RED web interface. At the top, there's a '使用者設置' (User Settings) section with a '關閉' (Close) button. Below it, a '顯示' (Show) section contains tabs for '節點' (Nodes) and '安裝' (Install). A '鍵盤' (Keyboard) section is also visible. The main area shows a search for 'random' with 17 / 3314 results. The 'Palette' on the left lists several nodes, with 'node-red-node-random' highlighted by a red box. The right sidebar shows the configuration for the 'random' node, including a 'Generate' dropdown set to 'a whole number - integer' and 'From'/'To' fields set to 1 and 10 respectively. The bottom status bar shows 'localhost:1880/#'.

使用者設置

關閉

顯示

節點

安裝

鍵盤

排序: a-z順序 日期順序

Q random 17 / 3314

Palette

random-generator_node-red-contrib

A package to meet the randomness generation needs of node-red developers

1.0.0 3 月前

node-red-node-random

A Node-RED node that when triggered generates random values.

0.4.0 3 月前

node-red-contrib-random-event-generator

localhost:1880/#

例題8- 編輯 random 節點

刪除 取消 完成

屬性

屬性 msg.payload

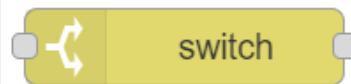
Generate a whole number - integer

From 1

To 10

名稱 名稱

功能-switch



switch

按屬性值來分配消息的傳送路線。

1_溫度亂數

switch

溫度

switch

編輯 switch 節點

刪除 取消 完成

屬性

名稱 名稱

屬性 msg. payload

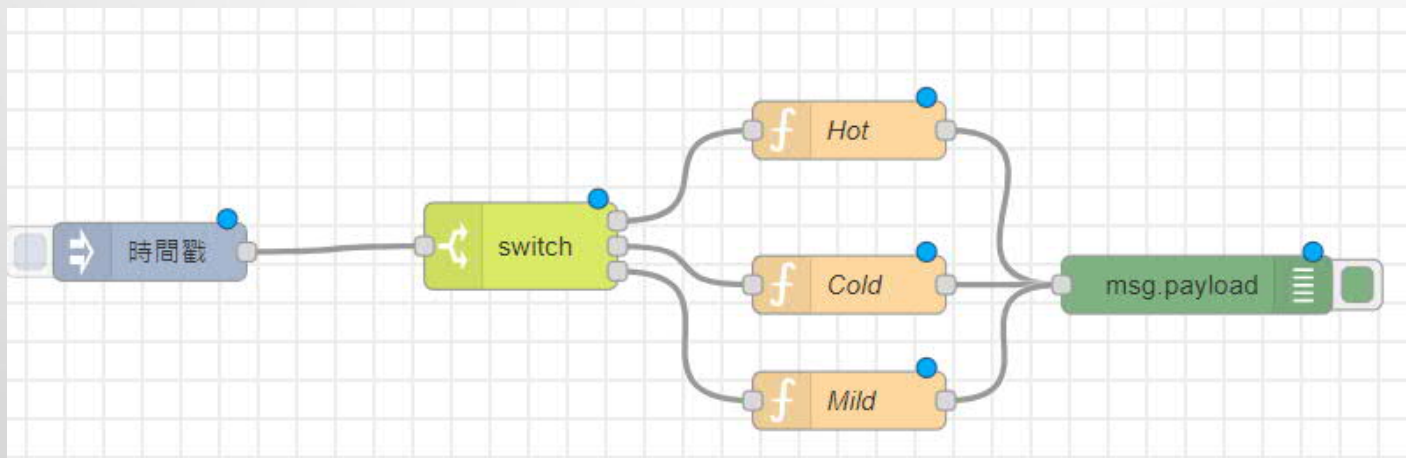
== a₂ → 1 ×

+ 添加

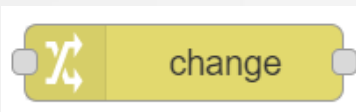
練習：

使用switch & function 節點做以下工作

1. 數字 <20 ，顯示溫度很冷，關掉風扇與冷氣
2. $28 > \text{數字} > 20$ ，顯示溫度剛好，開風扇、關掉冷氣
3. 數字 >28 ，顯示溫度很熱，開風扇與冷氣



功能-change



change

設置，更改，刪除或移動消息，流程上下文或全局上下文的屬性。

如果指定了多個規則，則將按定義的順序來應用它們。



設定屬性值

inject 節點送出的 payload 屬性值是數字 100, change 節點改成150



規則

設定	▼	▼ msg.payload	✕
到	▼ % 150		

+ 添加

修改屬性值

inject 節點送出的payload 屬性值是字串 This is a book.
change 節點取代 book 成為 pen



A screenshot of the configuration interface for the 'change' node. It features a dropdown menu set to '修改' (Change) with a downward arrow. To its right is a text input field containing 'msg.payload'. Below these, there is a search section with a magnifying glass icon and the label '搜索' (Search). The search input field contains 'a_z book' and has a close button (X). Below the search field, the label '替代為' (Replace with) is followed by another input field containing 'a_z pen'.

刪除屬性值

inject 節點送出的payload 屬性值後, 在 change 節點刪除 msg.payload 屬性, 因為屬性已經刪除, 所以 debug 節點顯示的是 undefined



轉移屬性

inject 節點送出的 payload 屬性值是數字 20, change 節點可以將 payload 轉移成 age

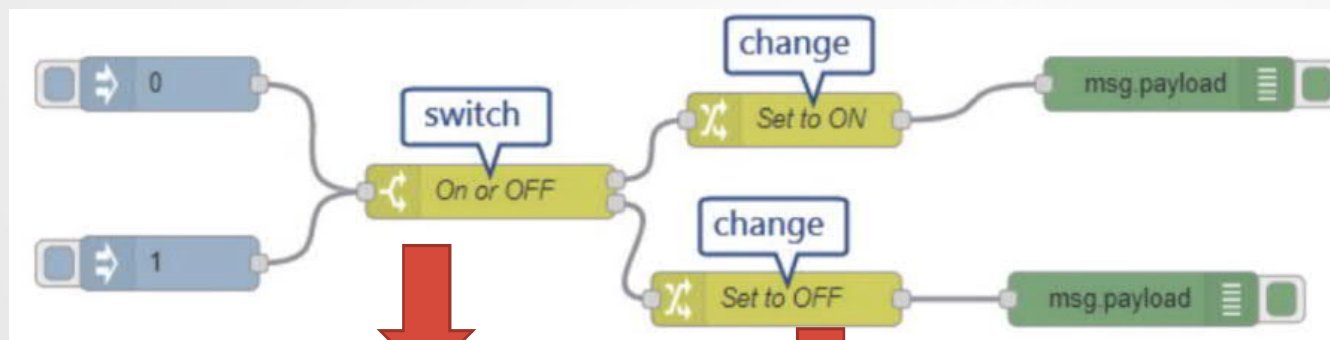


msg 物件的內容沒有 payload 屬性, 但有 age 屬性

```
msg : Object
  { _msgid: "41834fca.395b3", topic: "", age: 20 }
```

練習：switch + change

將 inject 節點送出的0和1,使用 switch 和 change 節點來分別輸出成 ON 和OFF



名稱: On or OFF

屬性: msg.payload **被比較的屬性**

規則:

- == 0 → 1
- == 1 → 2

選擇運算子 **比較值**

新增條件

全選所有規則

☐ 重建資料序列

名稱: Set to ON

規則:

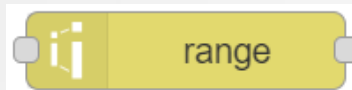
- 設定 msg.payload 到 ON

名稱: Set to OFF

規則:

- 設定 msg.payload 到 OFF

功能-range



range

將數值映射為另一個區間的數值

✓ 輸入

payload 數值

有效荷載一定得是一個數值。否則則會映射失敗。

✓ 輸出

payload 數值

被映射到新區間的數值。

編輯 range 節點

刪除

取消

完成

⚙ 屬性

⋮ 屬性

msg. payload

⦿ 操作

按比例msg.payload

➡ 映射輸入資料:

從: e.g. 0

到: e.g. 99

➡ 至目標範圍:

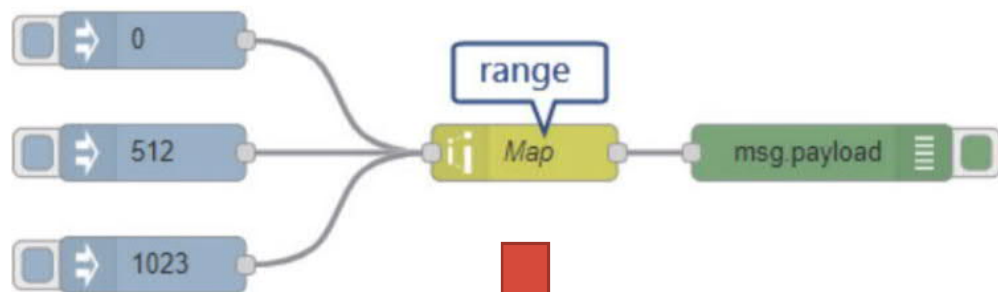
從: e.g. 0

到: e.g. 255

range 範圍節點可以將值從一個範圍對應至另一個範圍

練習：

使用range 節點將值0~1023 對應輸出成0~255



映射輸入資料:

從: 0 到: 1023

至目標範圍:

從: 0 到: 255

☒ 取最接近整數?

名稱: Map

功能-delay

delay

對通過節點的消息進行延遲發送或限制。

例題8-

延遲 5s

時間戳

編輯 delay 節點

刪除

取消

完成

屬性

行為設置

時長

名稱

延遲每一條資訊

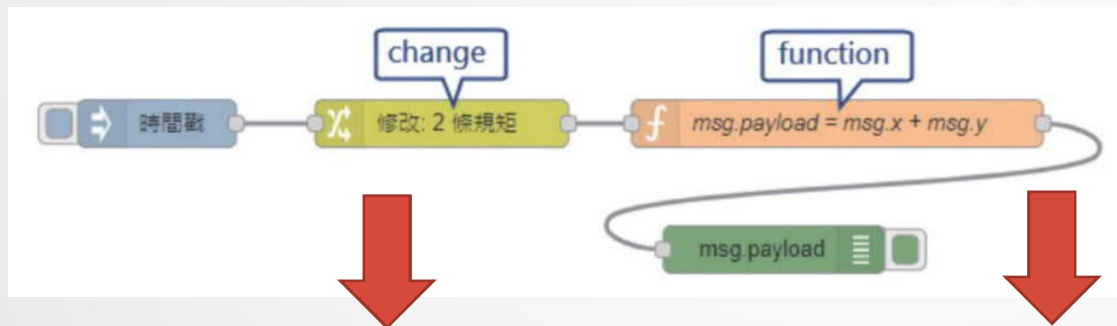
固定延遲時間

5 秒

名稱

練習：change + function

使用 change 節點指定x和y屬性的值, 然後使用 function 節點將x和y的值相加, 和顯示相加結果



Configuration panel for the 'change' node, showing two rules:

- Rule 1: 設定 (Setting) to msg.x, 到 (to) 0, 1.
- Rule 2: 設定 (Setting) to msg.y, 到 (to) 0, 2.

Configuration panel for the 'function' node, showing the code:

```
msg.payload = msg.x + msg.y;  
return msg;
```

The panel also includes a '名稱' (Name) field with the value 'msg.payload = msg.x + msg.y', and tabs for 'Setup', '設置' (Settings), '函數' (Function), and '關閉' (Close).

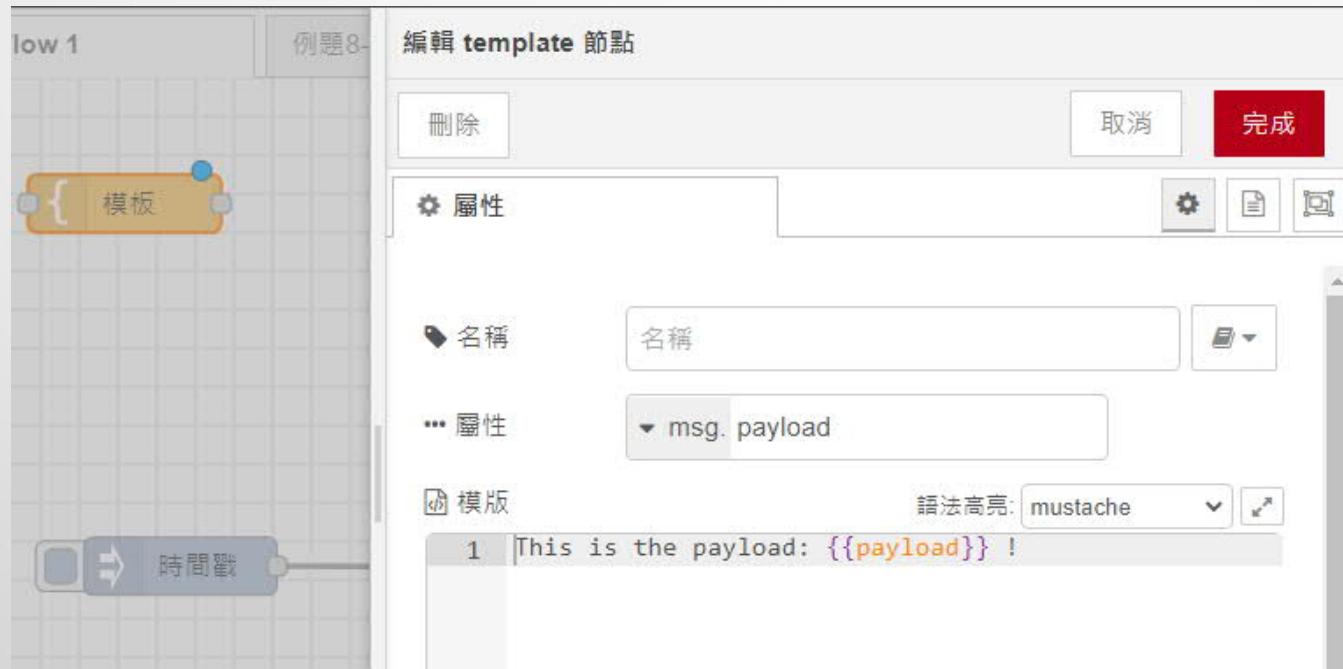
msg.payload : number

3

功能-template

template

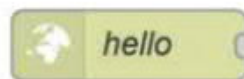
根據提供的模板設置屬性。



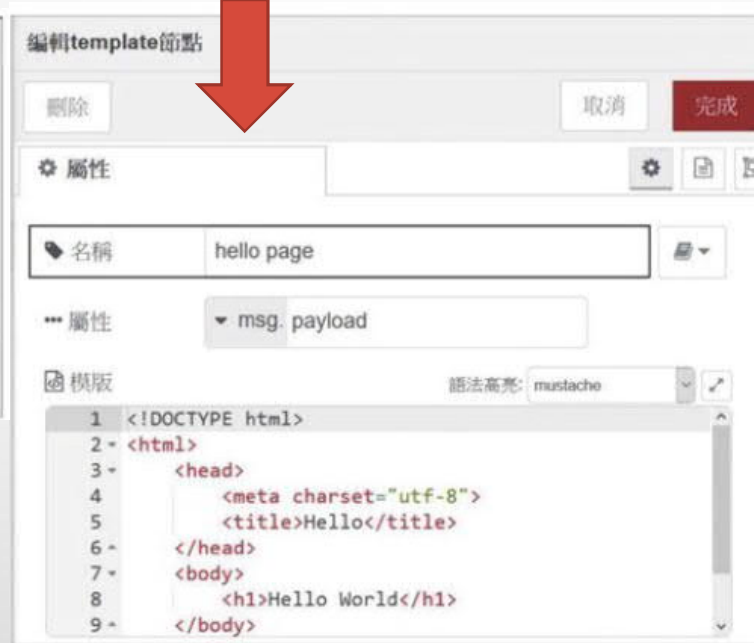
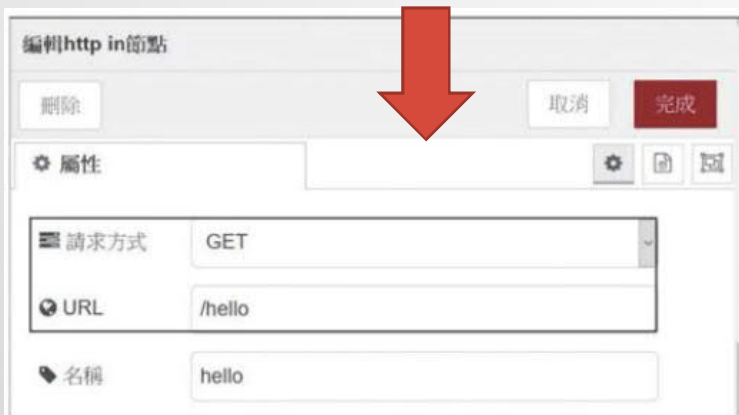
練習：

在Node-RED 中新增網頁

http in



http response



<http://localhost:1880/hello>





```
<!DOCTYPE html>  
<html>  
<head>  
<meta charset="utf-8">  
<title>Hello</title>  
</head>  
<body>  
<h1>Hello World</h1>  
</body>
```